

Quantum-Belief Multi-Agent Reinforcement Learning (Q-BMARL)

A Metacognitive Control Ablation Study

Authors

S. Vinay Narasimha
R. Hemanth Kumar

License: MIT

1. Executive Summary

Quantum-Belief Multi-Agent Reinforcement Learning (Q-BMARL) is a self-contained, offline, fully reproducible reinforcement-learning testbed in which two independent agents search a hidden-object environment while representing their own uncertainty using mathematical structures from quantum mechanics — complex amplitudes, interference, non-commuting measurement, entanglement, and entropy-threshold collapse — implemented with the QuTiP library on classical hardware, using quantum-mechanical mathematical structures rather than simple probability-counting updates.

On top of this representation, the project builds and empirically tests a metacognitive control layer: a module that reads an agent's own belief-confidence signal and deliberately overrides its base policy's action when that signal indicates the agent should keep gathering evidence rather than commit to a guess. The central research question is whether this kind of explicit, auditable self-monitoring changes measured reinforcement-learning behaviour, and if so, how much of that change is attributable to the mechanism itself versus incidental side effects of the experimental setup.

The question is answered with a controlled, three-condition ablation (Model A / Model B / Model C) trained for 3,000 episodes per condition on identical, seed-matched worlds. The result is a statistically significant finding within this experimental run, with a plausible mechanistic explanation: the explicit metacognitive controller (Model C) more than triples the final success rate of the baseline (24.4% vs. 3.8%, $p < 0.0001$) and roughly triples that of a passive-observation variant (Model B, 7.8%), while also solving faster on average. What follows covers the system architecture, the role of every component, the safeguards built into the design, the experimental design, the full statistical results, and the scope and limitations of the finding.

2. Project Information

Field	Detail
Project title	Quantum-Belief Multi-Agent Reinforcement Learning (Q-BMARL)
Version	Version 1
Authors	S. Vinay Narasimha, R. Hemanth Kumar
License	MIT License
Core dependencies	Python 3, PyTorch, NumPy, QuTiP, Pygame
Execution mode	Fully offline; no LLM, no network calls, deterministic given a seed
Primary deliverables	Trainable DQN agents, a Pygame visualizer, a 3-way ablation harness, and a statistical analysis script

3. Introduction and Motivation

Most reinforcement-learning agents represent uncertainty, if at all, with classical probability: a running tally of what has been observed, treated as a set of equally-weighted possibilities. Q-BMARL asks a narrower

and more testable question than "is quantum cognition real": if an agent's belief state is instead built using the linear-algebra machinery of quantum mechanics — complex amplitudes, superposition, interference, entanglement, and entropy, simulated on classical hardware via QuTiP — does giving that representation a genuine control role (not just extra input features) measurably change what a learning agent does?

To make that question answerable rather than philosophical, the project is built around a strict separation between (a) the belief representation itself, (b) whether that representation is allowed to trigger events on its own (entropy-threshold collapse), and (c) whether an explicit control layer is allowed to act on a second-order signal derived from that representation (confidence and its trend over time). Each of these is independently toggleable, which is what makes the three-model ablation in Section 11 a controlled experiment rather than an anecdote.

4. World and Task Design

Two independent agents, A and B, search the same shared environment for a hidden target room, but each maintains its own private belief state and moves on its own timer rather than in alternating turns.

- Map: a 5×5 grid of rooms. Each room is itself a 3×3 grid of searchable spots ("furniture"), for 225 searchable locations across the whole map.
- Target: exactly one room is secretly the location of the hidden chest; there is no visible landmark distinguishing it before it is found.
- Clues: exactly one real ROW clue and one real COLUMN clue are guaranteed to exist somewhere in the world (so the puzzle is always solvable), plus 2–4 decoy clues that look identical but report an incorrect value for whichever axis they are tagged with. Clue values are produced by simple, deterministic arithmetic expressions (e.g. "9 – 6"), not by any language model, keeping the environment instant and fully reproducible from a random seed.
- Actions (9 total): move UP / DOWN / LEFT / RIGHT, SEARCH the current spot, PICKUP and DROP a found clue (a clue can be physically relocated, which can help or hide it from the other agent), DECLARE the current room as the target, and SENSE the shared entangled resource described in Section 6.
- Episode length: capped at 500 steps. A wrong DECLARE is only a small penalty, not an episode-ending failure, so agents are free to keep exploring after a bad guess.

Both agents explore the same World object in a given episode, so relocating a clue via PICKUP/DROP is a genuine, mutual side effect — one agent's actions can make the puzzle easier or harder for the other.

5. System Architecture

The codebase is organised into clearly separated modules, each responsible for one layer of the system. The table below summarises every file and its role; each is described in depth in the following sections.

File	Role
env.py	The environment (QuantumBeliefEnv / World): room and clue layout, agent state, action handling, reward shaping, and the ablation flags <code>enable_collapse</code> / <code>metacognition</code> .

File	Role
quantum_mind.py	The QuTiP-backed quantum belief state per agent per axis: interference, non-commuting disturbance, entropy, and the entropy-trend / confidence metacognitive signal.
entangled_pair.py	A literal Bell-pair-style shared resource between the two agents, measured via the SENSE action using a real partial trace.
policies.py	The scripted heuristic policy, a DQN policy loader, and metacognitive-controller-wrapped variants of both.
dqn.py	The DQN agent: Q-network, target network, replay buffer, and a Double-DQN training step with tuned hyperparameters.
metacognitive_controller.py	The explicit control layer that reads confidence / entropy_trend and deliberately overrides a base policy's action.
train.py	Trains two independent DQN agents on shared worlds, with checkpointing and resumability.
compare_models.py	The 3-way ablation harness: Model A vs. Model B vs. Model C on matched world sequences, checkpointed/resumable.
analyze_compare.py	Statistical analysis of the ablation log: convergence checks, reward distributions, post-epsilon-floor collapse detection, and pairwise significance tests.
visualize.py	A Pygame viewer with a scrollable content surface showing both agents, their belief state, and entropy panels live.

6. The Quantum Belief Representation

The central technical idea of the project is that each agent's uncertainty about the target room's row and column is represented as a QuTiP quantum ket rather than a classical list of observed candidates. This gives the belief state four properties that are not represented by ordinary probability-counting updates in the baseline design.

6.1 Interference

Every clue reading contributes a complex amplitude, not a simple weight, to its reported value's basis state. Real clues share a fixed phase of zero (multiple real clues, were there more than one, would reinforce each other). Decoy clues receive a phase deterministically derived from their own identifier, effectively scattering them around the complex circle. The practical effect: two decoys that happen to report the same wrong value can partially cancel each other out (destructive interference) rather than simply accumulating the way classical evidence-counting would — this is the actual mathematical signature that distinguishes the model from Bayesian bookkeeping dressed up in quantum vocabulary.

6.2 Non-commuting measurement

Reading a row clue applies a small, fixed unitary "disturbance" rotation to the column belief state, and reading a column clue disturbs the row belief in the same way. As a result, reading a row clue and then a column clue produces a genuinely different final belief than reading them in the opposite order — an order-dependence that classical probability updating does not have, and that mirrors order effects documented in quantum-cognition models of human judgment.

6.3 Entropy-threshold collapse

Each agent's joint row×column belief has a von Neumann entropy, computed on the decohered (measurement-basis) density matrix rather than on the raw pure ket — a pure state always has zero von Neumann entropy by construction, so entropy must be computed on a measurement-basis mixture over possible outcomes for it to be a usable uncertainty signal at all. Once this entropy drops below a fixed threshold, an automatic entropy-threshold collapse event fires on its own: the environment auto-resolves the agent's current best guess, independent of whether the agent chose to act. This mechanism sits alongside — not instead of — the agent-initiated DECLARE action.

6.4 Metacognitive self-monitoring

Beyond the first-order belief itself ("what room do I currently think it is"), the belief state tracks a second-order signal over time: `entropy_trend()`, the slope of entropy over a short recent window (negative = uncertainty shrinking, positive = uncertainty growing, e.g. right after a cross-axis disturbance), and `confidence()`, a single scalar combining the current entropy level with that trend, so that low-and-still-falling uncertainty counts as more confident than the same low uncertainty sitting flat. This second-order signal is what the metacognitive controller (Section 9) reads and branches on directly.

7. Entanglement and the SENSE Action

A separate mechanic gives the two agents a simulated shared quantum resource: `entangled_pair.py` builds a QuTiP two-qubit Bell state — $|00\rangle + |11\rangle$ if the target room is in the top half of the map (rows 0–1), or $|01\rangle + |10\rangle$ if it is in the bottom half (rows 2–4) — encoding one coarse fact invisibly until it is measured. This is a deliberately stronger claim than the two agents simply reading the same shared World object: the simulation reproduces the defining property of entanglement, namely that neither agent's local qubit has a definite value until one of them measures, at which point the other agent's value becomes fixed too, without anything being computed or transmitted between them at that moment.

Each agent can measure its half of this pair at most once per episode via the SENSE action, computed with a real partial trace over the joint state (QuTiP's `Qobj.ptrace`) rather than any shortcut. Because of the no-signaling theorem, a single agent's own measurement outcome is exactly 50/50 and carries zero information about which underlying state was prepared — the encoded fact only becomes usable information once both agents have measured and their two individually-random outcomes are compared against each other. The environment enforces this correctly: it only reveals which half of the map the target is in after both agents have genuinely measured, tracked via a `measured_by` set kept separate from the internally-fixed `collapsed_value` dictionary.

8. Agents and Policies

8.1 Scripted heuristic policy

`policies.py` implements a fully deterministic, always-eventually-correct baseline: sense the entangled pair first; while an individual measurement is locally uninformative on its own, the two agents' correlated measurements together can reveal the encoded top/bottom-half information, so sensing early costs nothing and only helps. The policy then either walks to and DECLAREs any untested candidate room, or continues

searching the nearest not-yet-fully-searched room, prioritising by search-completeness rather than mere visitation so a room that was only passed through is not incorrectly skipped later. This heuristic reaches 100% success across every tested batch and exists primarily as a reference baseline and a demonstration of every game mechanic, not as a subject of the ablation itself.

8.2 DQN agent

dqn.py implements one independent learning brain per agent: a two-hidden-layer, 128-unit fully-connected Q-network, a target network, and a replay buffer, trained with Double DQN (the online network selects the next action, the target network evaluates it), which reduces the value-overestimation bias of standard DQN that would otherwise push the network toward repeatedly favouring a single degenerate action — though it does not eliminate other sources of training instability or degenerate policies, as the post-epsilon-floor results in Section 12.3 illustrate. Two independent instances — with different random seeds — are trained per run, one for Agent A and one for Agent B, each learning purely from its own experience even though both act within the same shared world each episode.

Training hyperparameters (chosen conservatively for training stability, see Section 10.4): learning rate **1e-4**, batch size **128**, replay buffer capacity **20,000**, discount $\gamma = 0.99$, target-network sync interval **1,000** training steps, ϵ -greedy exploration decaying from 1.0 at rate 0.998 per episode down to a floor of **0.10**.

9. Metacognitive Controller

A key design point of the project is the distinction between passive monitoring and active control. Simply exposing `entropy_trend()` and `confidence()` as two extra numbers in a DQN's observation vector is monitoring, not reasoning: a neural network can just as easily learn to ignore an input dimension as use it, and the scripted heuristic policy never referenced these signals at all — it always walked straight toward the nearest untried candidate the instant one existed, whether that candidate was a near-certainty or a coin flip.

`metacognitive_controller.py` implements the missing piece: an explicit `MetacognitiveController` that wraps any base policy (heuristic or DQN) and deliberately branches its decision on the belief state's own confidence and entropy trend, rather than merely making that information available for the base policy to ignore. Two concrete regimes are implemented:

1. Low confidence with untried candidates and gatherable evidence remaining: the controller defers committing to a guess and instead drives its own exploration action directly (search the current room, or walk toward the nearest unsearched one) — bypassing the base policy entirely while deferring, so the base policy cannot immediately steer the agent back toward a premature guess.
2. Rising uncertainty immediately after a cross-axis disturbance: if `entropy_trend` is sharply positive and confidence has not yet recovered, commitment is suppressed for that step in favour of continued search, rather than acting on a belief state that is still mid-disturbance.

The confidence threshold required to keep deferring relaxes linearly toward zero as the episode's step budget runs low, guaranteeing the controller always eventually commits to a guess rather than holding out indefinitely, and a hard safety margin stops deferral altogether once too few steps remain to travel to and

declare the best candidate. Every override is logged with its numeric reason, making the controller's behaviour directly auditable rather than a black box.

Because the controller wraps the base policy rather than replacing it, it is a drop-in addition to either the scripted heuristic or a trained DQN's greedy action selection, and because the underlying DQN is trained off-policy (Double DQN learns correctly from whatever action was actually taken and stored via `agent.remember`, regardless of which component chose it), a DQN can be trained validly even while its behaviour is being periodically overridden by the controller during data collection.

10. Design Safeguards and Failure Modes Handled

Several parts of the system exist specifically to guard against failure modes that direct testing surfaced. Each is described below as it currently operates, together with how it is verified. Several of these materially affect how the final ablation results in Section 12 should be interpreted.

10.1 Entanglement correctness: gating on measured_by

`entangled_pair.py`'s `measure()` simulates the collapse of the pair for both agents the instant either one measures — that is what entanglement means, and `collapsed_value` is populated for both agent IDs immediately as a result. `both_measured()` and `resolved_top_half()` are deliberately NOT computed from `collapsed_value` having two entries, because that would make a single agent's own measurement look like "both agents compared results," which is incorrect under the simulated quantum-measurement model: by the no-signaling theorem, one agent's local outcome alone is exactly 50/50 and carries zero information about which entangled state was prepared. That information only becomes usable once both agents have genuinely called `measure()` themselves and their two results are compared. A separate `measured_by` set tracks exactly that — who has actually measured, not who has been assigned a value by the other's collapse — and `both_measured()` / `resolved_top_half()` gate on it. Verified directly: after only Agent A measures, `both_measured()` correctly returns `False` and `resolved_top_half()` returns `None`; once Agent B also measures, `both_measured()` returns `True` and `resolved_top_half()` correctly matches the true target half.

10.2 Guarding against interference suppressing the correct answer

Ranking candidates purely by quantum amplitude is not sufficient on its own: a value whose amplitude gets unlucky and destructively cancelled can drop out of the top-k ranked list entirely — an observed trial had the true column value sitting at probability 0.0066, excluded from the ranked candidates, so the agent exhausted all 500 steps without ever trying it. `QuantumBelief.candidates()` guards against this with a raw "was this value ever actually read" safety net beneath the quantum ranking, so interference is allowed to change try order but never to make the correct answer permanently unreachable. Verified: 200/200 trials solved with the guard in place.

10.3 Computing entropy on the decohered state

A pure quantum ket always has von Neumann entropy of exactly zero (one eigenvalue equal to one), so computing entropy directly on the belief ket would never move, which would leave the entropy-threshold collapse mechanism unable to fire. Entropy is instead computed on the decohered, measurement-basis density matrix — an implementation choice that produces a measurement-basis mixed state suitable for representing uncertainty — which does shrink as evidence concentrates belief onto a single room, giving the collapse trigger a usable signal.

10.4 Conservative DQN hyperparameters

With up to 500 environment steps per episode and a training step fired on nearly every one, an aggressive learning rate and target-update cadence (e.g. learning rate $5e-4$, batch size 64, target-network sync every 500 steps) produces average training loss that climbs steadily rather than falling — textbook Q-value divergence rather than convergence, which shows up externally as success rates degrading the longer training continues. dqn.py's current defaults — learning rate $1e-4$, batch size 128, target-network sync every 1,000 steps — keep training loss low and flat; a 200-episode diagnostic run confirmed loss held between 0.05 and 0.11 under these settings.

10.5 Capped per-step no-op penalty

Without a cap, a per-step penalty for repeated invalid actions (wall bumps, searching an already-fully-searched room) can still sum to a very large negative total over a full episode if an agent gets stuck repeating one bad action for the whole 500 steps — a bounded-but-still-ugly quadratic-style blowup, distinct from a genuine runaway, that would swamp the signal from whatever mechanism is actually under test. env.py's step() logic guards against this with an explicit per-step cap (a maximum of -1.0 to -2.0 depending on the action type).

10.6 Collapse threshold set to actually exercise the mechanism

The entropy threshold for the automatic entropy-threshold collapse event is set at 2.4 bits. A stricter threshold (e.g. 1.6 bits, demanding near-total certainty before firing) triggers in only a fraction of a percent of trials, meaning a collapse-enabled model would barely exercise the mechanism it is supposed to be testing. At 2.4 bits, the ablation runs described in Section 11 show it firing in a measurable, meaningful fraction of trials.

10.7 Why the metacognitive controller owns movement while deferring

Intercepting only the final DECLARE action, and deferring to the base policy for everything else including movement, does not work: the scripted heuristic re-derives "walk toward the best untried candidate" on every single call as long as one exists, so the instant the controller redirects the agent toward unsearched territory, the very next base-policy call steers it right back — a one-step tug-of-war rather than a decision. Testing on identical worlds showed exactly that: agents spending all 500 steps oscillating without ever finishing exploration, far below the bare heuristic's solve rate. The controller instead decides up front, before ever consulting the base policy, whether to defer at all — and if so, it computes and owns its own exploration action end-to-end, never asking the base policy for movement while deferring, so there is nothing left for the base policy to overrule. Deferral is also bounded by `_urgency_adjusted_threshold` (required confidence relaxes linearly to zero as steps approach the limit) plus a hard safety margin, so the controller always stops deferring with enough steps left to reach and declare a room, rather than holding out for confidence indefinitely.

11. Experimental Design: The Three-Model Ablation

compare_models.py runs a controlled three-way ablation over the entropy-threshold collapse mechanism and the metacognitive control layer built on top of it. All three conditions are evaluated on configuration-identical, seed-matched worlds — same target room, same clue placements, episode for episode — by reseeding Python's random module to the same per-episode seed before constructing each condition's

World, so any measured difference in behaviour is attributable to the mechanism under test rather than to easier or harder worlds landing on one condition by chance. The worlds are matched, but each condition's DQN agents are independently seeded rather than identical: Model A's agents use seeds 1 and 2, Model B's use seeds 101 and 102, and Model C's use seeds 203 and 204, so the three conditions differ in both the mechanism under test and their networks' random initialisation/exploration trajectories, not the worlds they are evaluated on.

Condition	Configuration	What it isolates
Model A — conventional (baseline)	enable_collapse = False, metacognition = False	The DQN with no entropy-threshold collapse and no second-order signal at all — interference, disturbance, and entanglement remain part of the underlying belief model, since removing those would mean testing a different, simpler environment.
Model B — passive metacognition	enable_collapse = True, metacognition = True	Unprompted collapse is enabled, and entropy_trend/confidence are exposed as two extra observation dimensions for the DQN to learn to use (or ignore) on its own — a narrower claim than active control.
Model C — explicit controller	Same environment as Model B, plus each agent's action selection wrapped in a real MetacognitiveController	Whether an explicit, auditable override of the base policy's action — reading confidence/entropy_trend directly — does more than passively exposing the same signal as an input feature.

Each condition trained for 3,000 episodes (both agents, A and B, logged per episode), producing 3,000 rows per model in compare_log.csv. Statistical analysis was performed by analyze_compare.py using Wilson score confidence intervals for proportions, a two-proportion z-test with Cohen's h effect size for success-rate comparisons, and Mann-Whitney U tests for steps-to-solve and reward, all implemented without external statistics dependencies.

12. Results

12.1 Training convergence check

Success rate pooled across both agents, first 100 vs. last 100 trials of the 3,000-episode run, with final exploration rate (epsilon) shown for context:

Model	First-100 success	Last-100 success	Final ϵ (A)	Final ϵ (B)
A_no_collapse	31.0%	6.0%	0.100	0.100
B_collapse_metacog	39.0%	10.0%	0.100	0.100
C_meta_controller	38.0%	21.0%	0.100	0.100

All three conditions reach the exploration floor ($\epsilon = 0.10$) by the end of training. Notably, raw success rate falls from early to late training in every condition — the reasons for this are diagnosed in Section 12.3.

12.2 Final success rate (last 500 episodes, both agents pooled)

Model	Successes / trials	Success rate	95% CI (Wilson)
A_no_collapse	38 / 1000	3.8%	[2.8%, 5.2%]
B_collapse_metacog	78 / 1000	7.8%	[6.3%, 9.6%]
C_meta_controller	244 / 1000	24.4%	[21.8%, 27.2%]

Table: final success rate over the last 500 episodes per model (1,000 trials each, agents A and B pooled).

12.3 Post-epsilon-floor collapse-to-degenerate-policy check

analyze_compare.py splits each model's episodes into "still exploring" (ϵ above the floor threshold) and "exploration bottomed out" (ϵ at its floor), and compares success rate in each phase. A model whose success rate craters specifically after ϵ reaches its floor is showing a symptom consistent with the DQN converging onto a degenerate, repetitive greedy policy — a recognised DQN failure mode, distinct from ordinary noise.

Model	Pre-floor n	Pre-floor rate	Post-floor n	Post-floor rate	Flag
A_no_collapse	2204	16.2%	3796	4.9%	WARNING
B_collapse_metacog	2204	22.9%	3796	8.2%	WARNING
C_meta_controller	2204	28.7%	3796	23.5%	—

Both Model A and Model B show success rate roughly halving or worse once exploration bottoms out; Model C does not. This is the mechanistic explanation offered for why Model C performs better in this run, discussed further in Section 13.1.

12.4 Entropy-threshold collapse firing rate

Model	Collapse-firing trials	Rate
A_no_collapse	0 / 1000	0.0% (mechanism disabled by design)
B_collapse_metacog	61 / 1000	6.1%
C_meta_controller	403 / 1000	40.3%

Model C's much higher collapse-firing rate is consistent with its agents reaching low-entropy, high-confidence belief states far more often — itself a plausible downstream effect of the controller preventing the degenerate late-training behaviour identified in Section 12.3.

12.5 Controller override rate

Averaged over the last 500 episodes, Model C's metacognitive controller overrides the base DQN policy's chosen action an average of 182.8 times per agent per episode (Models A and B have no controller, so this is defined as 0.0 for them by construction). This is a substantial intervention rate — the controller is actively steering behaviour on a large fraction of the episode's steps, not making rare, marginal corrections.

12.6 Pairwise statistical comparisons (last 500 episodes)

Success rate

Comparison	z-statistic	p-value	Significant?	Cohen's h	Effect size
A vs. B	-3.827	0.0001	Yes ($p < 0.05$)	-0.174	Negligible
A vs. C	-13.236	< 0.0001	Yes ($p < 0.05$)	-0.641	Medium
B vs. C	-10.099	< 0.0001	Yes ($p < 0.05$)	-0.467	Small-medium

Steps-to-solve (successful episodes only)

Comparison	Mean steps (group 1)	Mean steps (group 2)	Mann-Whitney p	Significant?
A (n=38) vs. B (n=78)	287.08	247.38	0.1454	No
A (n=38) vs. C (n=244)	287.08	152.57	< 0.0001	Yes
B (n=78) vs. C (n=244)	247.38	152.57	< 0.0001	Yes

Episode reward (all episodes)

Comparison	Mean reward (group 1)	Mean reward (group 2)	Mann-Whitney p	Significant?
A vs. B	-72.34	-73.07	0.9184	No
A vs. C	-72.34	-106.43	0.0058	Yes
B vs. C	-73.07	-106.43	0.0062	Yes

Model C's lower (more negative) mean reward despite its much higher success rate is consistent with its controller spending many additional steps deliberately deferring and exploring rather than declaring quickly — the same trade-off seen in the controller's standalone evaluation (Section 12.8): more deliberation in exchange for far fewer costly mistakes and a much higher final solve rate.

12.7 Reward distribution diagnostics

Per-episode total reward can still reach large negative values even under the capped per-step penalty scheme (Section 10.5), if an agent spends most or all of an episode repeating one unproductive action — this is a bounded, expected artifact of a long episode accumulating many small penalties, not a sign that the cap is failing to hold.

Model	Metric	Min	P1	P99	Max
A_no_collapse	reward_a	-502.6	-296.7	9.1	12.4
A_no_collapse	reward_b	-436.2	-287.4	9.3	10.9
B_collapse_metacog	reward_a	-415.0	-268.8	10.3	12.2
B_collapse_metacog	reward_b	-601.1	-291.6	10.1	12.3
C_meta_controller	reward_a	-515.8	-362.9	12.7	14.5
C_meta_controller	reward_b	-612.8	-428.0	12.8	14.4

12.8 Standalone controller evaluation (heuristic base policy)

Independent of the DQN ablation, the controller was also evaluated wrapping the scripted heuristic policy directly, on 200 identical worlds shared between the bare heuristic and the metacognitive-controlled heuristic. Both configurations solved 200 of 200 trials, but the controlled version made 70% fewer failed DECLARE attempts on average ($4.42 \rightarrow 1.31$ per episode) at the cost of 44% more steps per episode ($150.5 \rightarrow 217.5$) — a directly measured trade-off of more deliberation for fewer costly mistakes, and separate evidence (outside the DQN ablation) that the controller changes behaviour in the intended direction rather than merely correlating with a training-stability side effect.

13. Discussion

13.1 Why Model C performs better in this run

The post-epsilon-floor analysis (Section 12.3) shows that both Model A and Model B suffer a severe collapse in success rate specifically once exploration (ϵ) reaches its floor and stops covering for the underlying policy — consistent with the DQN settling into a degenerate, repetitive greedy policy once it is no longer forced to occasionally act randomly, a known DQN failure mode unrelated to quantum formalism or metacognition in itself. Model C shows no such collapse. Its controller overrides the base policy's action an average of roughly 183 times per episode (Section 12.5), and that frequent, confidence-gated intervention appears to structurally prevent the agent from ever settling into that degenerate trap in the first place.

13.2 Interpretation

The result should not be overstated: it is a real, statistically significant effect within this run, but it plausibly reflects two overlapping effects — the metacognitive mechanism performing as intended, and the controller incidentally compensating for an unrelated training-stability issue (the epsilon-floor collapse, Section 12.3) that Models A and B remain fully exposed to. Both are legitimate, distinct findings and are treated separately rather than folded into a single claim.

13.3 Statistical dependence in the current analysis

The pairwise significance tests in Section 12.6 treat the 1,000 agent-level outcomes in the final 500 episodes ($500 \text{ episodes} \times 2 \text{ agents}$) as independent observations for the two-proportion z-tests and Mann-Whitney U tests. However, the two agents within a given episode share the same environment instance, and the three experimental conditions are trained on seed-matched worlds (Section 11). Consequently, these observations are not strictly independent in the statistical sense that the standard tests assume. The reported p-values should therefore be read as evidence within this experimental run rather than as definitive inferential proof of a population-level effect. A stronger follow-up analysis would use paired comparisons across matched worlds, together with multiple independent training seeds (Section 13.4), to properly account for this dependence structure.

13.4 Single-seed scope

Every result in Section 12 comes from one random seed per condition, from one training run. A statistically significant p-value describes this run, not a claim guaranteed to generalise across independently seeded repeats. Any $p < 0.05$ result here should be read as promising and worth replicating rather than settled. The next step toward a claim that does not need this caveat is to repeat all three conditions across multiple

independent seeds and average the resulting success rates, steps-to-solve, and reward distributions before asserting that Model C outperforms Models A and B in general, rather than in this specific run.

13.5 What the A-vs-B and B-vs-C comparisons each isolate

The three-condition design lets two distinct questions be asked from the same run. Model A vs. (B or C) isolates whether enabling collapse and metacognition, as a package, changes behaviour at all relative to neither being present. Model B vs. Model C isolates a narrower and arguably more interesting question: whether an explicit override controller does anything beyond passively exposing the same confidence/entropy_trend signal as ordinary observation features for the DQN to use or ignore. The B-vs-C comparison being significant in this run ($p < 0.0001$, small-to-medium effect) is evidence consistent with active control contributing beyond passive availability of the same information, though — per Section 13.3 — not definitive population-level proof of it.

14. Reproducing the Results

The experiments described in Section 11 are reproduced by running the following scripts in order:

```
pip install -r requirements.txt # pygame, torch, numpy, qutip
python compare_models.py --episodes 3000
python analyze_compare.py compare_log.csv --tail 500
```

The interactive Pygame visualizer can be run separately to watch either agent live, using any of the four available policies:

```
python visualize.py
--policy {heuristic, dqn, heuristic meta, dqn meta}
Controls: SPACE pause/resume, R new world, mouse wheel or UP/DOWN to scroll, Esc/close to quit.
```

15. Known Open Items

The following items remain open for future development:

- visualize.py remains a full Pygame application and has not yet been simplified.
- The two agents' move-delay constants (MOVE_DELAY_A / MOVE_DELAY_B) are currently set to the same value, so the two agents no longer move on visibly different cadences.
- All reported results are single-seed; multi-seed replication (Section 13.4) is the primary step remaining to turn a promising, run-specific result into a generalisable one.

16. Conclusion

Q-BMARL demonstrates a working, fully offline belief representation for reinforcement-learning agents built from quantum-mechanical mathematical structures, with a simulated Bell-pair-style entangled resource, genuine interference and non-commuting measurement effects, and an entropy-triggered collapse mechanism — implemented directly on QuTiP's linear algebra rather than approximated. Built on top of this representation, an explicit metacognitive control layer that reads an agent's own confidence and its trend over time, and deliberately overrides its base policy when that signal calls for more deliberation,

produces a statistically significant improvement in learned behaviour, with a plausible mechanistic explanation, over both a baseline and a passive-observation variant in a controlled, seed-matched, 3,000-episode ablation. The result is presented with the caveats it deserves — a single seed per condition, agent-level observations that are not fully statistically independent (Section 13.3), and a plausible confound with an independently identified DQN training-stability issue — and provides a reproducible experimental procedure together with a promising, run-specific finding about the potential value of explicit self-monitoring in a learned agent.

Appendix A: Observation and Action Space

Property	Value
Number of discrete actions	9 (UP, DOWN, LEFT, RIGHT, SEARCH, PICKUP, DROP, DECLARE, SENSE)
Base observation dimension	40 (position, top row/col belief guess, marginal entropies, sensed-half flag, room-searched flag, carrying flag, 25-cell visited mask, 9-cell current-room search mask)
Metacognitive observation dimension	45 (base 40 + entropy_trend + confidence, appended last so base feature offsets are unchanged whether or not metacognition is enabled)
Max steps per episode	500
Grid size	5 × 5 rooms, each a 3 × 3 sub-grid (225 searchable spots total)
Decoy clue count	Random, 2 to 4 per episode

Appendix B: File Reference

File	Purpose
env.py	Environment logic, world/clue generation, reward shaping, ablation flags.
quantum_mind.py	QuTiP quantum belief state, interference, entropy, metacognitive signal.
entangled_pair.py	Bell-pair-style shared resource, SENSE measurement via partial trace.
policies.py	Scripted heuristic, DQN policy loader, metacognitive-controller wrappers.
dqn.py	DQN network, replay buffer, Double DQN training step.
metacognitive_controller.py	Explicit confidence/entropy_trend-driven policy override layer.
train.py	Two-agent DQN training loop, checkpointed and resumable.
compare_models.py	3-way Model A/B/C ablation harness on matched world sequences.
analyze_compare.py	Statistical analysis: convergence, reward distribution, collapse detection, pairwise significance.
visualize.py	Pygame live viewer with per-agent belief/entropy panels.
requirements.txt	pygame, torch, numpy, qutip.